

GUJARAT TECHNOLOGICAL UNIVERSITY**BE- SEMESTER-VI EXAMINATION – WINTER 2025****Subject Code: 3160707****Date: 25-11-2025****Subject Name: Advanced Java Programming****Time: 02:30 PM TO 05:00 PM****Total Marks: 70****Instructions:**

1. Attempt all questions.
2. Make suitable assumptions wherever necessary.
3. Figures to the right indicate full marks.
4. Simple and non-programmable scientific calculators are allowed.

		MARKS
Q.1	(a) Describe the concept of JDBC.	03
	(b) Write a Web application using servlet to find whether entered number is Prime or not.	04
	(c) List the different types of JDBC drivers and explain type3 driver with its advantages and disadvantages.	07
Q.2	(a) What are the components of URL? Describe the use of URL class with an example.	03
	(b) Explain CallableStatement with example.	04
	(c) Write a client server program using TCP where client sends two numbers and server responds with the sum of them.	07
	OR	
	(c) Write a client server program using TCP where client send number and server respond reverse of given number.	07
Q.3	(a) What is session? List the different ways to manage the session.	03
	(b) Discuss servlet life cycle methods.	04
	(c) Explain use of ServletConfig and ServletContext object with example.	07
	OR	
Q.3	(a) How MVC architecture is better than conventional approach?	03
	(b) List and Explain various stages of JSP life cycle.	04
	(c) Write a Web application using servlet to find the sum of all the digits of an input integer.	07
Q.4	(a) Define the init() method of a servlet.	03
	(b) Write a servlet to display the name and age of a person received as parameters.	04
	(c) List out JSP Action Tags and explain it.	07
	OR	
Q.4	(a) How the GET method different from the POST method?	03
	(b) What is filter? List out the application of the filter.	04
	(c) Write a Web Application using JSP to get an integer N (where N>0) and display all the prime number up to N.	07
Q.5	(a) Discuss JSP Exception Handling.	03
	(b) List out JSF HTML tags and explain any two.	04
	(c) Draw the Hibernate Architecture diagram and explain each part in brief.	07

OR

- | | | |
|------------|--|-----------|
| Q.5 | (a) Explain any three Spring form tags with example. | 03 |
| | (b) What is JSF? List and explain its features. | 04 |
| | (c) Explain the steps to develop the Hibernate web application. | 07 |

Q.1 (a) Describe the concept of JDBC.

JDBC (Java Database Connectivity)

Definition:

JDBC is an **API (Application Programming Interface)** in Java that allows Java applications to **connect to and interact with relational databases**. It provides a standard way for Java programs to execute SQL statements, retrieve results, and manage database connections.

Concept and Working

1. Purpose:

- JDBC acts as a bridge between a Java application and a database.
- It allows Java applications to **perform CRUD operations** (Create, Read, Update, Delete) on database data.

2. Components of JDBC:

- **Driver Manager:** Manages a list of database drivers. Establishes a connection between Java and the database.
- **Connection:** Represents a session with a specific database.
- **Statement:** Used to execute SQL queries.
 - `Statement` – For static SQL queries.
 - `PreparedStatement` – For dynamic or parameterized queries.
 - `CallableStatement` – For calling stored procedures.
- **ResultSet:** Holds data retrieved from the database after executing a query.
- **SQLException:** Handles database-related errors.

3. How JDBC Works (Step-by-Step):

1. **Load the JDBC Driver:** Load the driver class for the database (e.g., MySQL, Oracle).
 2. `Class.forName("com.mysql.cj.jdbc.Driver");`
 3. **Establish a Connection:** Connect to the database using `DriverManager`.
 4. `Connection con = DriverManager.getConnection(`
 5. `"jdbc:mysql://localhost:3306/mydatabase", "username",`
`"password");`
 6. **Create a Statement:** Prepare a statement object to send SQL queries.
 7. `Statement stmt = con.createStatement();`
 8. **Execute SQL Query:** Execute queries using `executeQuery` (for SELECT) or `executeUpdate` (for INSERT/UPDATE/DELETE).
 9. `ResultSet rs = stmt.executeQuery("SELECT * FROM students");`
 10. **Process Result:** Loop through `ResultSet` to fetch data.
 11. `while(rs.next()) {`
 12. `System.out.println(rs.getString("name"));`
 13. `}`
 14. **Close Connection:** Release database resources.
 15. `rs.close();`
 16. `stmt.close();`
 17. `con.close();`
-

Key Features of JDBC

- Platform-independent (works on any OS supporting Java).
- Supports multiple databases using different drivers.
- Provides transaction management (commit, rollback).
- Can execute SQL statements dynamically or statically.

(b) Write a Web application using servlet to find whether entered number is Prime or not.

1. HTML Form (`index.html`)

```
<!DOCTYPE html>
<html>
<head>
    <title>Prime Number Checker</title>
</head>
<body>
    <h1>Check Prime Number</h1>
    <form action="PrimeServlet" method="post">
        Enter a number:
        <input type="number" name="number" required>
        <input type="submit" value="Check">
    </form>
</body>
</html>
```

2. Servlet Code (`PrimeServlet.java`)

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/PrimeServlet")
public class PrimeServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        try {
            int number = Integer.parseInt(request.getParameter("number"));
            boolean isPrime = true;

            if (number <= 1) {
                isPrime = false;
            } else {
                for (int i = 2; i <= Math.sqrt(number); i++) {
                    if (number % i == 0) {
                        isPrime = false;
                        break;
                    }
                }
            }
        }
    }
}
```

```

        }
    }

    out.println("<html><body>");
    out.println("<h2>Number Entered: " + number + "</h2>");

    if (isPrime) {
        out.println("<h3>" + number + " is a Prime Number.</h3>");
    } else {
        out.println("<h3>" + number + " is NOT a Prime
Number.</h3>");
    }

    out.println("<a href='index.html'>Check Another Number</a>");
    out.println("</body></html>");

    } catch (NumberFormatException e) {
        out.println("<h3>Please enter a valid integer!</h3>");
    }
}
}

```

(c) List the different types of JDBC drivers and explain type3 driver with its advantages and disadvantages.

JDBC Drivers – Detailed Explanation

JDBC (Java Database Connectivity) allows Java applications to connect with databases. To communicate with databases, **different types of drivers** have been developed, each with its own architecture and working mechanism.

Types of JDBC Drivers

1. **Type 1 – JDBC-ODBC Bridge Driver**
 - Uses **ODBC drivers** to connect Java applications to databases.
 - Partly Java, partly native.
 - **Disadvantages:** Platform-dependent, needs ODBC setup, not recommended for production.
 2. **Type 2 – Native API Driver**
 - Converts JDBC calls into **database-specific calls** using native libraries provided by the database vendor.
 - Faster than Type 1 but **platform-dependent**.
 3. **Type 3 – Network Protocol / All-Java Driver**
 - Converts JDBC calls into a **database-independent network protocol**.
 - Middleware server translates this protocol to the **database-specific protocol**.
 - Entirely written in Java, making it platform-independent.
 4. **Type 4 – Thin / Direct-to-Database Java Driver**
 - Converts JDBC calls directly into the **database's native protocol**.
 - Pure Java, no middleware required.
 - Very fast and commonly used in production.
-

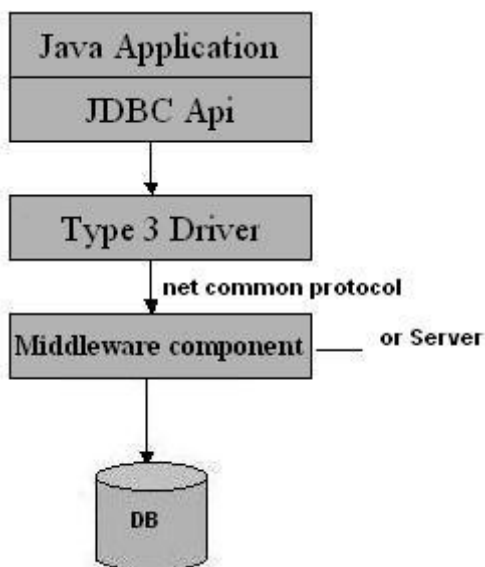
Type 3 JDBC Driver – Detailed Overview

Definition:

Type 3 driver is also known as a **Network Protocol driver**. It is a **pure Java driver** that uses a **middleware server** (application server) to communicate with the database. The client sends requests in a **database-independent protocol**, and the middleware converts them into **database-specific SQL calls**.

- **Client Side:** Java application + Type 3 driver
 - **Server Side (Middleware):** Converts protocol → native database calls
 - **Database Side:** Actual database executes SQL
-

Architecture of Type 3 Driver



Explanation:

1. **Client:** Sends JDBC requests using Type 3 driver.
 2. **Middleware Server:** Receives database-independent protocol requests, translates them to database-specific SQL commands.
 3. **Database:** Executes the SQL commands and returns results.
 4. **Middleware:** Converts the result set back to client-understandable format and sends it to Java application.
-

Working Steps of Type 3 Driver

1. **Load Driver:** Java application loads Type 3 JDBC driver.
2. `Class.forName("com.xyz.jdbc.Driver");`
3. **Establish Connection:** Connect to middleware server instead of the database directly.
4. `Connection con = DriverManager.getConnection(`

5. `"jdbc:xyz://middleware-server:port/dbname", "username", "password");`
 6. **Create Statement:** Java application prepares SQL queries using `Statement` or `PreparedStatement`.
 7. **Execute Query:** Type 3 driver sends the query to middleware server in a **database-independent protocol**.
 8. **Middleware Processing:** Middleware server converts query to the **specific database protocol** and executes it.
 9. **Retrieve Result:** Middleware sends results back to Java application.
 10. **Close Connection:** Release resources once done.
-

Advantages of Type 3 Driver

1. **Platform Independent:** Pure Java driver; runs on any platform supporting Java.
 2. **Database Independent:** Middleware handles database-specific translation; switching databases does not affect client.
 3. **Network-Friendly:** Optimized for distributed environments and internet applications.
 4. **Centralized Management:** Middleware server allows easier management of multiple clients and connections.
 5. **Better Security:** Middleware can implement access control and encryption.
-

Disadvantages of Type 3 Driver

1. **Middleware Dependency:** Requires middleware server setup and maintenance.
2. **Slightly Slower than Type 4:** Because every request passes through middleware.
3. **Complexity:** More layers in architecture make debugging slightly harder.
4. **Cost:** Middleware server may require licensing or additional resources.

Q.2 (a) What are the components of URL? Describe the use of URL class with an example.

1. What is a URL?

URL (Uniform Resource Locator) is the address of a resource on the Internet. It specifies **where a resource is located** and **how it can be accessed**.

Example URL:

`https://www.example.com:8080/docs/index.html?user=admin#top`

2. Components of a URL

A URL is made up of several parts, each having a specific role.

General Format of URL

`protocol://host:port/path?query#fragment`

1. Protocol

- Specifies the **communication method** used to access the resource.
- Common protocols:
 - http
 - https
 - ftp
 - file
 - mailto

Example:

`https`

2. Host (Domain Name / IP Address)

- Identifies the **server** where the resource is located.
- Can be a **domain name** or **IP address**.

Example:

`www.example.com`

3. Port Number

- Specifies the **port** on which the server is listening.
- Optional (default ports are used if not specified).
 - HTTP → 80
 - HTTPS → 443

Example:

`8080`

4. Path

- Indicates the **location of the resource** on the server.
- Can represent a directory or file.

Example:

```
/docs/index.html
```

5. Query String

- Sends **data to the server** in key–value pairs.
- Begins with ?
- Multiple parameters are separated by &

Example:

```
?user=admin
```

6. Fragment (Reference)

- Points to a **specific section** within a resource.
- Begins with #
- Used mainly in HTML pages.

Example:

```
#top
```

Uses of URL Class

- To create URL objects.
 - To extract URL components like protocol, host, port, file, etc.
 - To read data from web resources.
 - Used in networking and web-based Java applications.
-

4. Common Constructors of URL Class

```
URL(String spec)
```

```
URL(String protocol, String host, String file)
```

```
URL(String protocol, String host, int port, String file)
```

5. Common Methods of URL Class

Method	Purpose
<code>getProtocol()</code>	Returns protocol name
<code>getHost()</code>	Returns host name

Method	Purpose
<code>getPort()</code>	Returns port number
<code>getPath()</code>	Returns file path
<code>getQuery()</code>	Returns query string
<code>getFile()</code>	Returns file name
<code>getRef()</code>	Returns fragment
<code>openStream()</code>	Opens input stream to read data

6. Example Using URL Class

```
import java.net.URL;

public class URLEDemo {
    public static void main(String[] args) {
        try {
            URL url = new URL(

"https://www.example.com:8080/docs/index.html?user=admin#top"
            );

            System.out.println("Protocol: " + url.getProtocol());
            System.out.println("Host: " + url.getHost());
            System.out.println("Port: " + url.getPort());
            System.out.println("Path: " + url.getPath());
            System.out.println("Query: " + url.getQuery());
            System.out.println("File: " + url.getFile());
            System.out.println("Reference: " + url.getRef());

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output

```
Protocol: https
Host: www.example.com
Port: 8080
Path: /docs/index.html
Query: user=admin
File: /docs/index.html?user=admin
Reference: top
```

(b) Explain CallableStatement with example.

CallableStatement

CallableStatement is an interface in JDBC that is used to **call stored procedures** and **stored functions** present in a database. Stored procedures are pre-compiled SQL statements that are stored inside the database and can be executed whenever required. By using CallableStatement, a Java application can execute complex database logic efficiently.

CallableStatement is a **sub-interface of PreparedStatement**, which means it supports parameterized queries. It allows both **input parameters (IN)** and **output parameters (OUT)**, making it useful when a database procedure returns values after execution.

Working of CallableStatement

When a Java program wants to execute a stored procedure, it first establishes a database connection. Then it creates a CallableStatement object using the `prepareCall()` method of the Connection interface. The SQL syntax for calling a stored procedure uses curly braces {}. Input values are passed using setter methods, and output values are retrieved using getter methods after execution.

CallableStatement improves performance because stored procedures are **compiled once in the database** and reused multiple times. It also improves security and maintainability since the database logic is separated from application logic.

Example

Stored Procedure in Database (MySQL)

```
CREATE PROCEDURE getSquare(IN num INT, OUT result INT)
BEGIN
    SET result = num * num;
END;
```

This procedure takes one input value and returns its square.

Java Program Using CallableStatement

```
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;

public class CallableDemo {
    public static void main(String[] args) {
        try {
```

```

        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/testdb", "root", "password");

        CallableStatement cs =
            con.prepareCall("{call getSquare(?, ?)}");

        cs.setInt(1, 5); // IN parameter
        cs.registerOutParameter(2, java.sql.Types.INTEGER); // OUT
parameter

        cs.execute();

        int square = cs.getInt(2);
        System.out.println("Square is: " + square);

        cs.close();
        con.close();

    } catch (Exception e) {
        System.out.println(e);
    }
}
}

```

(c) Write a client server program using TCP where client sends two numbers and server responds with the sum of them.

Server Program (TCP Server)

```

import java.net.ServerSocket;
import java.net.Socket;
import java.io.DataInputStream;
import java.io.DataOutputStream;

public class SumServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server started, waiting for client...");

            Socket s = ss.accept();
            System.out.println("Client connected");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new
DataOutputStream(s.getOutputStream());

            int num1 = dis.readInt();
            int num2 = dis.readInt();

            int sum = num1 + num2;
            dos.writeInt(sum);

            s.close();
            ss.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

```
    }  
    }  
}
```

Client Program (TCP Client)

```
import java.net.Socket;  
import java.io.DataInputStream;  
import java.io.DataOutputStream;  
  
public class SumClient {  
    public static void main(String[] args) {  
        try {  
            Socket s = new Socket("localhost", 5000);  
  
            DataOutputStream dos = new  
DataOutputStream(s.getOutputStream());  
            DataInputStream dis = new DataInputStream(s.getInputStream());  
  
            int a = 10;  
            int b = 20;  
  
            dos.writeInt(a);  
            dos.writeInt(b);  
  
            int result = dis.readInt();  
            System.out.println("Sum received from server: " + result);  
  
            s.close();  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

Working Explanation

First, the server starts and waits for a client connection on port **5000**.
The client connects to the server and sends two integers.
The server reads these integers, calculates their sum, and sends the result back.
Finally, the client receives the sum and displays it on the console.

Output

Client side output:

```
Sum received from server: 30
```

OR

(c) Write a client server program using TCP where client send number and server respond reverse of given number.

Server Program (TCP Server)

```
import java.net.ServerSocket;
import java.net.Socket;
import java.io.DataInputStream;
import java.io.DataOutputStream;

public class ReverseServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server started... Waiting for client");

            Socket s = ss.accept();
            System.out.println("Client connected");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new
DataOutputStream(s.getOutputStream());

            int num = dis.readInt();
            int rev = 0;

            while (num != 0) {
                int digit = num % 10;
                rev = rev * 10 + digit;
                num = num / 10;
            }

            dos.writeInt(rev);

            s.close();
            ss.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client Program (TCP Client)

```
import java.net.Socket;
import java.io.DataInputStream;
import java.io.DataOutputStream;

public class ReverseClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);

            DataOutputStream dos = new
DataOutputStream(s.getOutputStream());
            DataInputStream dis = new DataInputStream(s.getInputStream());

            int number = 12345;
            dos.writeInt(number);

            int reversed = dis.readInt();
            System.out.println("Reversed number received: " + reversed);

            s.close();
        }
    }
}
```

```
        } catch (Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```

Working Explanation

First, the server starts and listens on port **5000**.

The client connects to the server and sends a number.

The server reads the number, reverses it using a loop, and sends the reversed number back.

Finally, the client receives and displays the reversed number.

Output

Client side output:

```
Reversed number received: 54321
```

Q.3 (a) What is session? List the different ways to manage the session.

Session

A **session** is a way to maintain **state information** between a client and a server across multiple requests.

In web applications, the HTTP protocol is **stateless**, which means each request is treated as independent. A session helps the server **remember user-specific data** such as login details, preferences, or shopping cart contents during a user's interaction with the website.

A session starts when a user first accesses a web application and ends when the user logs out, closes the browser, or after a predefined **timeout period**.

Need for Session Management

Since HTTP does not remember previous requests, session management is required to:

- Track user login status
 - Store temporary user data
 - Maintain continuity across multiple pages
 - Provide personalized user experience
-

Ways to Manage Session

There are several techniques used to manage sessions in web applications:

1. Cookies

Cookies are small pieces of data stored on the **client's browser**.

The server sends a cookie to the client, and the browser sends it back with every subsequent request.

- Used to store session ID or user data
- Simple and widely supported
- Limited storage size

Example use: remembering login status.

2. URL Rewriting

In URL rewriting, session information is appended to the URL as a parameter.

Example:

```
http://example.com/home;jsessionid=12345
```

- Useful when cookies are disabled
 - Session ID travels with every request
 - Less secure as session ID is visible in URL
-

3. Hidden Form Fields

Session data is stored inside **hidden fields** of HTML forms and passed from one request to another.

```
<input type="hidden" name="sessionId" value="12345">
```

- Works only with form submissions
 - Not suitable for complex applications
-

4. HttpSession (Server-Side Session)

`HttpSession` is a Java servlet API feature that stores session data on the **server side**.

- Session object is created automatically
- Data is stored securely on server
- Client only holds session ID (usually via cookie)

Example use: storing user objects or login data.

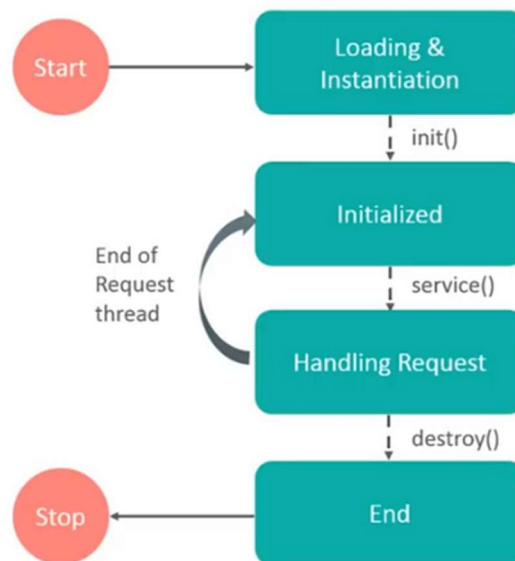
5. SSL Session

SSL session uses secure HTTPS connections to maintain session information.

- Highly secure
- Used in sensitive applications like banking
- Performance overhead due to encryption

(b) Discuss servlet life cycle methods.

Servlet Life Cycle



The **servlet life cycle** refers to the complete process through which a servlet passes from the moment it is loaded into memory until it is removed from the server. This entire life cycle is managed by the **servlet container** (such as Apache Tomcat, GlassFish, or WebLogic). The developer does not control the life cycle directly; instead, the container invokes specific methods at different stages.

1. Loading and Instantiation

When a client sends a request to a servlet for the first time, the servlet container checks whether the servlet class is already loaded.

If it is not loaded, the container loads the servlet class and creates a **single instance** of the servlet. This instance is shared among multiple client requests, which makes servlets efficient in terms of memory usage.

2. Initialization Phase – `init()` Method

After the servlet instance is created, the container calls the `init()` method exactly **once** during the servlet's lifetime.

This method is used to perform **initial setup operations**. Typical tasks include reading configuration parameters, initializing variables, or establishing database connections. Since `init()` is called only once, it is not suitable for handling client requests.

This phase prepares the servlet so that it can efficiently handle incoming requests.

3. Request Processing Phase – `service()` Method

Once the servlet is initialized, it is ready to handle client requests. Every time a client sends a request, the servlet container calls the `service()` method.

This is the **most frequently executed** method in the servlet life cycle.

The `service()` method processes the request and generates the response. It also determines the type of HTTP request (such as GET or POST) and delegates the request to the appropriate handler method.

Multiple client requests can be handled concurrently using the same servlet instance, making servlets highly scalable.

4. Destruction Phase – `destroy()` Method

When the server is shut down or the servlet is removed from service, the container calls the `destroy()` method.

This method is executed only **once**, just before the servlet is destroyed. It allows the servlet to release any resources it has acquired during its lifetime, such as database connections, files, or network resources.

After the `destroy()` method completes execution, the servlet instance becomes eligible for garbage collection.

(c) Explain use of ServletConfig and ServletContext object with example.

ServletConfig and ServletContext

In a Java web application, servlets often need **configuration information** and access to **application-wide resources**. For this purpose, the Servlet API provides two important objects: **ServletConfig** and **ServletContext**.

Both objects are created by the **servlet container**, but they differ in **scope and usage**.

1. ServletConfig

ServletConfig is an object that contains **initialization parameters specific to a particular servlet**.

Each servlet has its **own ServletConfig object**, and it is used to pass configuration data to that servlet at the time of initialization.

ServletConfig helps in separating **configuration information** from servlet logic.

Uses of ServletConfig

- To read servlet-specific initialization parameters
 - To access servlet name
 - To get the ServletContext object
-

Example using ServletConfig

web.xml configuration:

```
<servlet>
  <servlet-name>MyServlet</servlet-name>
  <servlet-class>MyServlet</servlet-class>

  <init-param>
    <param-name>adminName</param-name>
    <param-value>Komal</param-value>
  </init-param>
</servlet>

<servlet-mapping>
  <servlet-name>MyServlet</servlet-name>
  <url-pattern>/myservlet</url-pattern>
</servlet-mapping>
```

Servlet code using ServletConfig:

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletConfig;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class MyServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        ServletConfig config = getServletConfig();
        String name = config.getInitParameter("adminName");

        out.println("Admin Name: " + name);
    }
}
```

2. ServletContext

ServletContext is an object that represents the **entire web application**.

It is **shared among all servlets** in the same application and is created only **once** when the application starts.

ServletContext is mainly used to store and share **global data** and access **application-level resources**.

Uses of ServletContext

- To share data between multiple servlets
- To read application-wide initialization parameters
- To access resources like files, images, or database information
- To perform logging

Example using ServletContext

web.xml configuration:

```
<context-param>
```

```
<param-name>companyName</param-name>
<param-value>ABC Technologies</param-value>
</context-param>
```

Servlet code using ServletContext:

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class ContextServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        ServletContext context = getServletContext();
        String company = context.getInitParameter("companyName");

        out.println("Company Name: " + company);
    }
}
```

OR

Q.3 (a) How MVC architecture is better than conventional approach?

In early web applications, developers commonly used the **conventional approach**, where **presentation logic (HTML/UI)**, **business logic**, and **data access code** were written together in a single program or page (for example, JSP pages containing Java code). While this approach worked for small applications, it created many problems as applications grew in size and complexity.

The **MVC (Model–View–Controller) architecture** was introduced to overcome these limitations by clearly separating responsibilities.

Conventional Approach (Without MVC)

In the conventional approach:

- All logic is tightly coupled.

- JSP or servlet handles database operations, business rules, and UI generation together.
- Any small change in logic or UI often affects the entire code.
- Code becomes difficult to read, test, debug, and maintain.

As the application grows, this approach leads to **spaghetti code**, making development and maintenance time-consuming and error-prone.

MVC Architecture

MVC divides the application into three independent components:

- **Model:** Handles business logic and data (database interaction).
 - **View:** Responsible for presentation (UI).
 - **Controller:** Acts as an intermediary, handling user requests and coordinating between Model and View.
-

Why MVC is Better than the Conventional Approach

1. Separation of Concerns

MVC clearly separates business logic, presentation, and request handling. Each component has a specific role, which makes the application well-structured and organized. In contrast, the conventional approach mixes everything in one place.

2. Better Maintainability

In MVC, changes in one layer (such as UI design) do not affect business logic. This makes maintenance easier and reduces the risk of introducing errors. In the conventional approach, even small changes require modifying large sections of code.

3. Improved Reusability

Models can be reused across multiple views, and controllers can manage different views using the same business logic. In the conventional approach, reuse is minimal because logic is tightly bound to presentation.

4. Enhanced Scalability

MVC supports large-scale applications because developers can work independently on different layers. Conventional architecture struggles as the application size increases.

5. Easier Testing and Debugging

Since components are separated, testing becomes easier. Business logic can be tested independently of the UI. Debugging is also simpler compared to the conventional approach where all logic is intertwined.

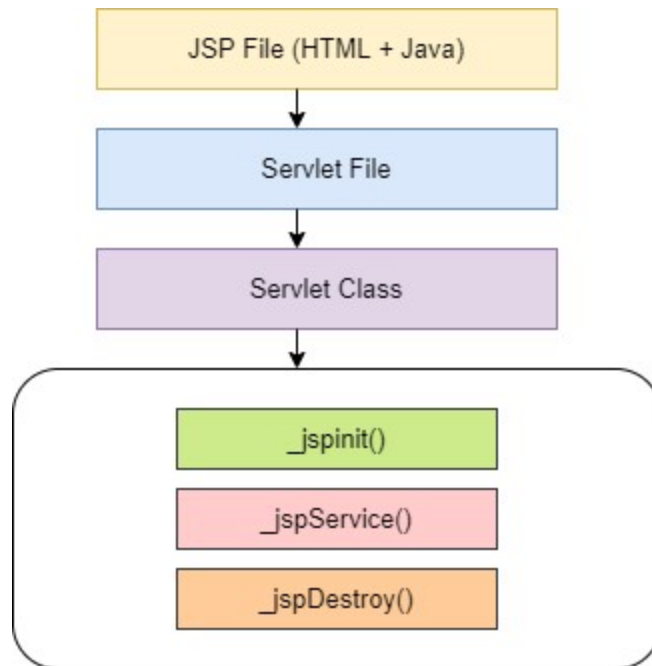
6. Better Team Collaboration

MVC allows multiple developers to work simultaneously—UI designers focus on Views, backend developers focus on Models, and system architects focus on Controllers. This is difficult in the conventional approach.

7. Cleaner and More Readable Code

MVC promotes clean, structured, and readable code. The conventional approach often results in cluttered and hard-to-understand programs.

(b) List and Explain various stages of JSP life cycle.



JSP Life Cycle Stages

The **JSP life cycle** defines how a JSP page is translated, compiled, executed, and eventually removed by the server. The diagram shows the key stages:

1. Translation Phase (JSP → Servlet)

- The **JSP file**, which contains HTML and Java code, is **first translated into a servlet** by the JSP engine.
- The resulting servlet is a `.java` file that extends `HttpServlet`.
- This translation is automatic and happens **only once** unless the JSP file is modified.

2. Compilation Phase (Servlet File → Servlet Class)

- The generated servlet file is then **compiled into a servlet class** (`.class` file).
- This compiled servlet is now a standard Java class capable of handling HTTP requests.

=

3. Initialization Phase (`_jspInit()` method)

- When the servlet class is **loaded into memory**, the container calls the `_jspInit()` method (similar to `init()` in regular servlets).
 - This method initializes resources needed by the JSP, such as database connections or configuration parameters.
 - Executed **once** during the life of the JSP servlet.
-

4. Request Handling Phase (`_jspService()` method)

- For **every client request**, the container calls the `_jspService()` method.
 - This method contains the main logic of the JSP, including HTML output and Java code execution.
 - Handles multiple requests concurrently.
-

5. Destruction Phase (`_jspDestroy()` method)

- When the JSP servlet is removed or the server shuts down, the container calls `_jspDestroy()`.
- This method is used to **release resources** like database connections, file handles, or session objects.
- Executed **once** before the JSP is destroyed.

(c) Write a Web application using servlet to find the sum of all the digits of an input integer.

1. HTML Form (`index.html`)

```
<!DOCTYPE html>
<html>
<head>
  <title>Sum of Digits</title>
</head>
<body>
  <h1>Calculate Sum of Digits</h1>
  <form action="SumDigitsServlet" method="post">
    Enter a number:
    <input type="number" name="number" required>
    <input type="submit" value="Calculate">
  </form>
</body>
```


</html>

2. Servlet Code (SumDigitsServlet.java)

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/SumDigitsServlet")
public class SumDigitsServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        try {
            int number = Integer.parseInt(request.getParameter("number"));
            int sum = 0;
            int temp = number;

            while (temp != 0) {
                sum += temp % 10;
                temp /= 10;
            }

            out.println("<html><body>");
            out.println("<h2>Number Entered: " + number + "</h2>");
            out.println("<h3>Sum of Digits: " + sum + "</h3>");
            out.println("<a href='index.html'>Try Another Number</a>");
            out.println("</body></html>");

        } catch (NumberFormatException e) {
            out.println("<h3>Please enter a valid integer!</h3>");
        }
    }
}
```

3. Sample Output

Input: 467

Output:

```
Number Entered: 467
Sum of Digits: 17
```

Q.4 (a) Define the init() method of a servlet.

The `init()` method in a servlet is a **lifecycle method** defined in the `javax.servlet.Servlet` interface. It is called by the servlet container (like Tomcat) **once when the servlet is first loaded into memory**, before it starts handling any client requests. Its main purpose is to perform **initialization tasks**, such as setting up resources, reading configuration parameters, or establishing database connections.

Key Points about `init()`:

1. **Called only once:** When the servlet is loaded for the first time. It is **not called for every request**.
2. **Purpose:** Initialize the servlet and its resources.
3. **Parameters:** It can take a `ServletConfig` object as a parameter (`init(ServletConfig config)`), which allows access to **initialization parameters** defined in the deployment descriptor (`web.xml`).
4. **Throws:** Can throw `ServletException` if initialization fails.

Example:

```
import javax.servlet.*;
import java.io.*;

public class MyServlet extends GenericServlet {
    private String message;

    public void init() throws ServletException {
        // Initialization code
        message = "Hello, this servlet is initialized!";
    }

    public void service(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>" + message + "</h1>");
    }
}
```

(b) Write a servlet to display the name and age of a person received as parameters.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class PersonServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
        response.setContentType("text/html");

        String name = request.getParameter("name");
        String age = request.getParameter("age");

        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Person Details</h2>");
        if(name != null && age != null) {
            out.println("Name: " + name + "<br>");
            out.println("Age: " + age);
        } else {
            out.println("Please provide 'name' and 'age' as request parameters.");
        }
        out.println("</body></html>");
    }
}
```

```
protected void doPost(HttpServletRequest request, HttpServletResponse
response)

    throws ServletException, IOException {

    doGet(request, response);

}

}
```

Output:

```
Person Details
Name: Het
Age: 25
```

(c) List out JSP Action Tags and explain it.

1. `<jsp:useBean>`

- Used to **create or locate a JavaBean object**.
- If the bean already exists in the specified scope, it is reused; otherwise, a new bean is created.

Syntax:

```
<jsp:useBean id="obj" class="package.ClassName" scope="request" />
```

2. `<jsp:setProperty>`

- Used to **set the value of a property** of a JavaBean.
- Property values are usually taken from request parameters.

Syntax:

```
<jsp:setProperty name="obj" property="propertyName" value="value" />
```

3. `<jsp:getProperty>`

- Used to **retrieve and display the value** of a JavaBean property.

Syntax:

```
<jsp:getProperty name="obj" property="propertyName" />
```

4. *<jsp:include>*

- Used to **include another resource** (JSP, HTML, or servlet) **at runtime**.
- Changes in the included file are reflected without recompiling the main JSP.

Syntax:

```
<jsp:include page="header.jsp" />
```

5. *<jsp:forward>*

- Used to **forward the request and response** to another resource.
- Control does not return to the original JSP after forwarding.

Syntax:

```
<jsp:forward page="next.jsp" />
```

6. *<jsp:param>*

- Used to **pass parameters** to another resource.
- Commonly used inside `<jsp:include>` or `<jsp:forward>`.

Syntax:

```
<jsp:param name="user" value="admin" />
```

7. *<jsp:plugin>*

- Used to **embed applets or JavaBeans** into a JSP page.
- It automatically generates the required browser-specific HTML code.

Syntax:

```
<jsp:plugin type="applet" code="MyApplet.class" width="300" height="200" />
```

OR

Q.4 (a) How the GET method different from the POST method?

The **GET** and **POST** methods are HTTP request methods used to send data from a client (browser) to a server. They differ in the way data is sent, security, and usage.

GET Method	POST Method
Data is sent through the URL as query string.	Data is sent in the request body .
Data is visible in the browser address bar.	Data is not visible in the address bar.
Less secure because data is exposed in the URL.	More secure compared to GET.
Limited amount of data can be sent.	Large amount of data can be sent.
Data can be bookmarked .	Data cannot be bookmarked.
Data is stored in browser history.	Data is not stored in browser history.
Mainly used for retrieving data .	Mainly used for submitting or updating data .
Can be cached by the browser.	Cannot be cached by default.
Suitable for non-sensitive data.	Suitable for sensitive data like passwords.

Example

GET Method URL:

`http://localhost:8080/app/login?username=admin&password=123`

POST Method:

- Data is sent internally and not shown in the URL.

(b) What is filter? List out the application of the filter.

A **Filter** in a Java web application is a **server-side component** provided by the Servlet API that is used to **intercept requests and responses** before they reach a servlet or after they leave a servlet. Filters are mainly used to perform **pre-processing and post-processing tasks** without modifying the servlet code.

Filters are defined by implementing the `javax.servlet.Filter` interface and are configured in `web.xml` or by annotations.

Applications (Uses) of Filters

1. Authentication and Authorization

Filters are used to check whether a user is logged in or has proper permissions before allowing access to a resource.

2. Logging and Auditing

Filters can record request details such as IP address, URL accessed, date, and time for monitoring and debugging.

3. Input Validation

Filters validate request data before it reaches the servlet to ensure correctness and security.

4. Data Compression

Filters can compress responses before sending them to the client to improve performance.

5. Encoding Management

Filters ensure consistent character encoding (e.g., UTF-8) for requests and responses.

6. Security Filtering

Filters help protect applications from attacks such as SQL injection, XSS, and CSRF by filtering malicious input.

7. Response Modification

Filters can modify the response content or headers before it is sent to the client.

8. Performance Monitoring

Filters measure execution time of requests to analyze application performance.

(c) Write a Web Application using JSP to get an integer N (where $N > 0$) and display all the prime number up to N.

JSP Page: `prime.jsp`

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head>
    <title>Prime Numbers</title>
</head>
<body>

<h2>Display Prime Numbers Up to N</h2>

<form method="post">
    Enter N:
    <input type="number" name="n" min="1" required>
    <input type="submit" value="Show Primes">
</form>
```

```

<br>
<%
    String value = request.getParameter("n");

    if (value != null) {
        int n = Integer.parseInt(value);

        out.println("<h3>Prime numbers up to " + n + " :</h3>");

        for (int i = 2; i <= n; i++) {
            boolean isPrime = true;

            for (int j = 2; j <= i / 2; j++) {
                if (i % j == 0) {
                    isPrime = false;
                    break;
                }
            }

            if (isPrime) {
                out.print(i + " ");
            }
        }
    }
%>
</body>
</html>

```

Sample Output

If user enters:

N = 10

Output:

Prime numbers up to 10:
2 3 5 7

Q.5 (a) Discuss JSP Exception Handling.

JSP Exception Handling

JSP Exception Handling is the mechanism used to handle **runtime errors** that occur during the execution of a JSP page. JSP provides a built-in way to manage exceptions so that errors can be handled gracefully without crashing the application or showing technical error messages to users.

Need for Exception Handling in JSP

- Prevents abnormal termination of JSP pages
 - Displays user-friendly error messages
 - Helps in debugging and maintenance
 - Improves reliability of web applications
-

Ways to Handle Exceptions in JSP

1. Using `errorPage` and `isErrorPage` Attributes

This is the **most common and recommended method**.

- `errorPage` attribute: Specifies the JSP page that will handle exceptions.
- `isErrorPage` attribute: Indicates that a JSP page is an error-handling page.

Syntax:

Main JSP Page:

```
<%@ page errorPage="error.jsp" %>
```

Error Handling JSP Page:

```
<%@ page isErrorPage="true" %>
```

- When an exception occurs, control is automatically forwarded to `error.jsp`.
 - The error page has access to the **implicit object** `exception`.
-

2. Using Try-Catch Block in JSP Scriptlet

Exceptions can be handled directly using Java's `try-catch` blocks inside JSP scriptlets.

Example:

```
<%  
    try {  
        int a = 10 / 0;  
    } catch (Exception e) {  
        out.println("Error: " + e.getMessage());  
    }  
%>
```

- Useful for handling **specific logic errors**.
 - Not recommended for large applications as it mixes Java code with JSP.
-

3. Using *web.xml* Error Handling

Errors can be handled **centrally** using deployment descriptor.

Example:

```
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/error.jsp</location>
</error-page>
```

- Handles exceptions across the entire application.
 - Provides better control and reusability.
-

Implicit Object Used in JSP Exception Handling

- **exception**
Available only in pages with `isErrorPage="true"`.
It holds the reference to the thrown exception.
-

Advantages of JSP Exception Handling

- Clean separation of normal and error logic
- Centralized error management
- Improved user experience
- Easier debugging and maintenance

(b) List out JSF HTML tags and explain any two.

List of JSF HTML Tags

Some commonly used JSF HTML tags are:

- `<h:form>`
- `<h:inputText>`
- `<h:inputSecret>`
- `<h:outputText>`
- `<h:commandButton>`
- `<h:commandLink>`
- `<h:selectOneMenu>`
- `<h:selectOneRadio>`
- `<h:selectManyCheckbox>`
- `<h:selectManyListbox>`

- `<h:outputLabel>`
 - `<h:messages>`
 - `<h:message>`
 - `<h:panelGrid>`
 - `<h:panelGroup>`
 - `<h:graphicImage>`
-

Explain Any Two JSF HTML Tags

1. `<h:inputText>`

- Used to create a **text input field** in a JSF page.
- It allows users to enter text data.
- The entered value is bound to a **managed bean property**.

Example:

```
<h:inputText value="#{userBean.name}" />
```

Explanation:

This tag renders an HTML `<input type="text">` field. The value entered by the user is automatically stored in the `name` property of `userBean`.

2. `<h:commandButton>`

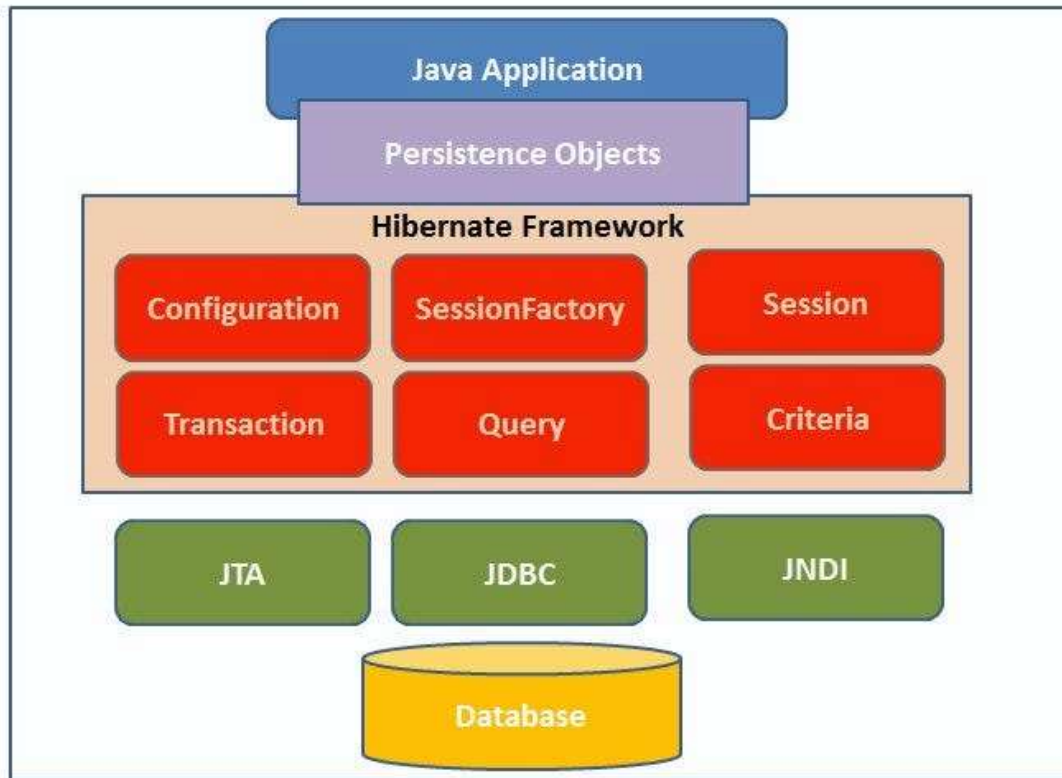
- Used to create a **submit button** in JSF.
- It triggers an **action method** in a managed bean when clicked.

Example:

```
<h:commandButton value="Submit" action="#{userBean.submit}" />
```

(c) Draw the Hibernate Architecture diagram and explain each part in brief.

Hibernate Architecture



What is Hibernate?

Hibernate is a **Java-based Object Relational Mapping (ORM) framework** that is used to develop **database-independent persistence logic**.

It allows Java developers to work with **Java objects instead of SQL queries**. Hibernate automatically converts **object operations (save, update, delete, fetch)** into **SQL statements** and executes them on the database.

Hibernate reduces:

- Boilerplate JDBC code
- Manual SQL writing
- Database dependency

It improves **performance, maintainability, and scalability** of applications.

Hibernate Architecture – Detailed Explanation of Each Component

1. Java Application

This is the **client layer** of Hibernate architecture.
It contains:

- Business logic
- Service classes
- DAO (Data Access Object) classes

The Java application **interacts with Hibernate APIs** instead of directly communicating with the database.

2. Persistence Objects

Persistence objects are **POJO (Plain Old Java Objects)** that represent **database tables**.

- Each class → one table
- Each object → one row
- Class fields → table columns

These classes are mapped using:

- XML mapping files (`.hbm.xml`) or
- Annotations (`@Entity`, `@Table`, etc.)

Hibernate manages these objects and makes them persistent.

3. Hibernate Framework

This is the **core layer** that handles mapping, caching, transactions, and query execution.

(a) Configuration

The **Configuration object** is used to:

- Load Hibernate configuration file (`hibernate.cfg.xml`)
- Read database connection details
- Load mapping information

It acts as the **starting point of Hibernate**.

Responsibilities:

- Database URL
 - Username & password
 - Dialect
 - Mapping classes
-

(b) SessionFactory

- SessionFactory is a **factory class** used to create Session objects.
- It is **heavyweight, thread-safe**, and created **only once** during application startup.

Responsibilities:

- Manages connection pool
 - Stores mapping metadata
 - Improves performance using caching
-

(c) Session

- Session represents a **single interaction** between application and database.
- It is **lightweight** and **not thread-safe**.
- Every database operation happens through Session.

Responsibilities:

- Save object
 - Update object
 - Delete object
 - Fetch object
-

(d) Transaction

- Transaction ensures **data consistency**.
- All operations inside a transaction either:
 - **Commit successfully**, or
 - **Rollback** in case of failure

It supports **ACID properties** of database systems.

(e) Query

- Query interface is used to execute **HQL (Hibernate Query Language)**.
- HQL works with **Java objects and properties**, not database tables.

Advantages:

- Database independent
 - Object-oriented
 - Easy to maintain
-

(f) Criteria

- Criteria API is used to create **dynamic and flexible queries**.
- No query strings are required.
- Conditions can be added programmatically.

It is useful when query conditions change at runtime.

4. JTA (Java Transaction API)

- JTA is used for **distributed transactions**.
- Supports transaction management across:
 - Multiple databases
 - Multiple resources

Used mainly in **enterprise-level applications**.

5. JDBC

- Hibernate internally uses **JDBC** to communicate with the database.
 - Developers do **not write JDBC code manually**.
 - Hibernate handles connection, statements, and result sets.
-

6. JNDI (Java Naming and Directory Interface)

- Used to **lookup database resources** like DataSource.
 - Common in application servers (Tomcat, WebLogic, JBoss).
-

7. Database

- The database stores actual application data.
 - Hibernate automatically generates SQL queries and executes them on the database.
-

OR

Q.5 (a) Explain any three Spring form tags with example.

1. <form:form>

<form:form> is the **main and mandatory tag** in Spring MVC forms. It creates an HTML <form> element and connects it with a **command (model) object** provided by the controller.

This tag:

- Acts as a **container** for all other Spring form tags
- Automatically binds form data to a Java object
- Manages HTTP method (GET/POST) and form action

It eliminates the need to manually read request parameters.

Example

```
<form:form action="register" method="post" modelAttribute="user">
    Name: <form:input path="name" /><br><br>
    <input type="submit" value="Register">
</form:form>
```

2. <form:input>

<form:input> is used to create a **text input field** and bind it to a specific property of the model object.

It replaces the traditional <input type="text"> tag and automatically handles data binding.

This tag:

- Displays existing values from the model
 - Automatically updates the model on form submission
 - Avoids writing request.getParameter() code
-

Example

```
<form:form modelAttribute="user">
    Username:
    <form:input path="username" />
</form:form>
```

3. <form:password>

<form:password> is used to create a **password input field** in a Spring form. It hides user input and securely binds the value to a model object property.

This tag:

- Generates `<input type="password">`
 - Prevents password from being displayed on reload
 - Integrates with validation mechanisms
-

Example

```
<form:form modelAttribute="user">
    Password:
    <form:password path="password" />
</form:form>
```

(b) What is JSF? List and explain its features.

JSF (JavaServer Faces) is a **Java-based web application framework** used to develop **component-based user interfaces** for Java web applications. It is part of the **Java EE (now Jakarta EE)** platform and follows the **MVC (Model–View–Controller)** architecture.

In JSF:

- **Model** → JavaBeans / Managed Beans
- **View** → XHTML pages using JSF tags
- **Controller** → JSF framework itself

JSF simplifies web application development by providing **reusable UI components**, **event handling**, **data binding**, and **validation**.

Features of JSF (List and Explain)

1. Component-Based Architecture

JSF provides a rich set of **reusable UI components** such as input fields, buttons, tables, and menus.

- Components are server-side objects
 - Automatically rendered as HTML
 - Promotes reusability and consistency
-

2. MVC Architecture Support

JSF strictly follows the **MVC design pattern**.

- Model: Managed Beans
- View: XHTML pages
- Controller: JSF lifecycle

This separation improves maintainability and scalability.

3. Managed Beans

JSF uses **Managed Beans** to handle application logic.

- Plain Java classes
 - Automatically managed by JSF container
 - Supports different scopes (request, session, application, view)
-

4. Event Handling

JSF supports **event-driven programming**.

- User actions (button clicks, value changes) trigger events
- Events are handled using listener methods in managed beans

This makes JSF applications interactive and dynamic.

5. Automatic Data Binding

JSF automatically binds **UI components to JavaBean properties**.

- No need to manually extract request parameters
 - Simplifies form processing
 - Reduces boilerplate code
-

6. Built-in Validation

JSF provides **built-in validators** such as:

- Required
- Length
- Range
- Regex

Custom validators can also be created easily.

7. Navigation Handling

JSF provides simple **page navigation management**.

- Navigation rules can be defined in `faces-config.xml`
- Or handled through annotations
- Makes page flow easy to control

8. Expression Language (EL) Support

JSF uses **Expression Language (EL)** to access managed bean properties and methods.

- Example: `#{user.name}`
- Reduces Java code inside view pages
- Improves readability

(c) Explain the steps to develop the Hibernate web application.

Steps to Develop a Hibernate Web Application

Developing a **Hibernate web application** involves a series of well-defined steps to integrate Hibernate with a Java web project. These steps ensure proper configuration, object-relational mapping, and database interaction in a structured and maintainable way.

1. Create a Web Project

- Create a **Dynamic Web Project** using an IDE such as Eclipse or IntelliJ.
- Configure the project with a web server like **Apache Tomcat**.
- Add required **Hibernate libraries (JAR files)** to the project.

2. Configure the Database

- Choose the database (MySQL, Oracle, PostgreSQL, etc.).
- Create a database and required tables.
- Note down database details such as:
 - JDBC URL
 - Username
 - Password

- Driver class
-

3. Create Hibernate Configuration File

- Create `hibernate.cfg.xml`.
- This file contains:
 - Database connection details
 - Hibernate dialect
 - SQL display settings
 - Mapping class details

Purpose:

It initializes Hibernate and connects the application to the database.

4. Create Persistent (Entity) Classes

- Create **POJO classes** that represent database tables.
- Each class field corresponds to a table column.
- Use **annotations** or mapping XML files to map classes with tables.

Purpose:

Enables object-relational mapping between Java classes and database tables.

5. Create Mapping Information

- Define mapping using:
 - Annotations (`@Entity`, `@Id`, `@Column`, etc.)
 - OR XML mapping files (`.hbm.xml`)

Purpose:

Tells Hibernate how Java objects are stored in database tables.

6. Build SessionFactory

- Use `Configuration` object to load `hibernate.cfg.xml`.
- Create a single **SessionFactory** object.
- SessionFactory is created once and reused throughout the application.

Purpose:

Acts as a factory for creating Session objects.

7. Open Session and Begin Transaction

- Open a Session from SessionFactory.
- Begin a transaction before performing database operations.

Purpose:

Ensures data consistency and integrity.

8. Perform Database Operations

- Use Session methods such as:
 - `save()`
 - `update()`
 - `delete()`
 - `get()` / `load()`
- Use **HQL or Criteria API** to fetch data.

Purpose:

Handles CRUD operations without writing SQL queries.

9. Commit Transaction and Close Session

- Commit the transaction if operations are successful.
 - Rollback in case of exception.
 - Close the session to release resources.
-

10. Integrate Hibernate with Web Layer

- Use Servlets, JSP, or Spring MVC to handle client requests.
- Call Hibernate logic from DAO classes.
- Display data using JSP pages.

Purpose:

Connects user interface with Hibernate backend.

11. Handle Exceptions and Logging

- Use try-catch blocks to handle Hibernate exceptions.
- Enable logging using Log4j or SLF4J.

Purpose:

Improves error handling and debugging.

12. Test and Deploy the Application

- Test application functionality.
- Deploy the project on a web server.
- Verify database operations.